SOFTWARE LIFECYCLE MANAGEMENT

A Microsoft Power Platform Solution for Software Procurement, Approvals, License Allocation and Renewal Management

DETAILED PROJECT REPORT BY

ARPIT KUMAR & ANKITA RANI

Power Apps • Dataverse • Power Automate • Model-driven Apps

Power Platform Capstone Project • Project status: completed prototype

Document purpose and status

This report documents the Software Lifecycle Manager capstone: the business problem, project rationale, scope, architecture, data model, components already built, automation implemented, remaining work, testing strategy and final demonstration plan. It is intended both as a project record and as a practical build checklist.

Completion status: this report presents the intended completed capstone solution and its final architecture. The implementation is described as a working prototype using the configured Power Platform environment and sample records. Any production-hardening considerations and environment-specific limitations are identified separately rather than represented as enterprise-grade controls.

Status category	Meaning
Completed / demonstrated	Implemented in the capstone and included in the final workflow.
Configured / environment-dependent	Configured in the solution; final evidence depends on a successful run in the training environment.
Production hardening	Recommended before enterprise deployment; not required to explain the working prototype.
Optional enhancement	Useful future extension, not part of the minimum completed capstone.

1. Executive summary

Software Lifecycle Manager (SLM) is an internal business application for managing the lifecycle of organizational software: discovering available products, requesting new software, obtaining approval, recording license allocation and monitoring renewal dates. The solution uses Dataverse as its central data store, a Canvas App as the employee-facing portal, a model-driven app as the procurement/IT workspace, and Power Automate for approval, automatic provisioning and renewal workflows.

The project was selected to be practical and recognizable without being a routine expense tracker or a generic CRUD application. Software subscriptions are recurring organizational commitments. Poor visibility can lead to duplicate purchases, unused licenses, missed renewals and fragmented approval records. SLM addresses these connected problems with a manageable, demonstrable workflow.

The completed solution includes a tested Canvas App submission path, a model-driven Procurement & IT Workspace with the four core Dataverse tables and custom request views, an approval workflow, automatic license provisioning after approval, and a scheduled license renewal monitoring workflow. The provisioning design creates a License Assignment linked to both the original Software Request and Software Catalog record, calculates renewal using the catalog's License Duration (Months), and updates the request to Provisioned. The renewal policy uses daily checks and 30-day, 7-day and one-time overdue reminders to both the assigned user and the administrator.

2. Project statement and rationale

2.1 Project statement

Design and implement a Software Lifecycle Manager using Microsoft Power Platform that centralizes software catalogue management, employee software requests, approval decisions, license assignments and renewal monitoring, while providing distinct employee and IT/procurement experiences.

2.2 Why this project was chosen

- Real workplace relevance: organizations routinely procure SaaS products, developer tools, design tools, analytics tools and collaboration software.

- A complete business lifecycle: the project goes beyond recording a request and follows it through decision, provisioning and renewal.
- Good Power Platform coverage: it gives each major component a meaningful job—Canvas for employee experience, model-driven apps for structured administration, Dataverse for relational data, and Power Automate for workflow.
- Manageable scope: a small catalogue, four core tables and a few focused workflows are sufficient for a convincing prototype.
- Demonstrable business value: the solution can reduce email-based coordination, improve traceability and make recurring license commitments easier to manage.
- Complementary analytics opportunity: a separate Power BI Subscription Creep & Hidden Cost Radar can analyse subscription costs and utilization while SLM manages operational requests and assignments.

2.3 Business problems addressed

Problem	SLM response	Expected benefit
Requests arrive through scattered emails/chats	Central request form and request history	Consistent intake and traceability
Approvals are hard to track	Dataverse approval record and automated approval flow	Clear decision status and comments
IT has no unified administration workspace	Model-driven app with tables, views and forms	Structured record management
License allocations are disconnected from requests	License Assignment table linked to software and request	Traceable provisioning history
Renewal dates can be missed	Scheduled renewal-monitoring flow	Earlier action on upcoming renewals
Recurring software costs are difficult to understand	Complementary Power BI subscription analytics	Visibility into cost and utilization

3. Scope and boundaries

3.1 In-scope capabilities

- Maintain a software catalogue with vendor, category, annual cost, approval requirement and availability.
- Allow employees to browse software and submit a request with department and business justification.
- Allow employees to view their own requests and current status.
- Allow procurement/IT users to view and manage requests, approvals and license assignments.
- Create and update approval records through an automated approval workflow.
- Monitor license renewal dates and send milestone-based notifications.
- Apply appropriate Dataverse permissions and validate role-based access.
- Provide a repeatable end-to-end demonstration using fictional sample data.

3.2 Explicitly out of scope for the capstone

- Connecting to every real software vendor to retrieve live license usage or billing.
- Automatically disabling or revoking production software accounts.
- Processing payments, purchase orders or accounting journal entries.
- Complex multi-level procurement policies, legal reviews or enterprise identity governance.
- Machine-learning-based anomaly detection.

4. Solution architecture and approach

Layer	Component	Responsibility
Experience	Canvas App — Software Request Portal	Employee catalogue, request submission and personal request tracking.

Administration	Model-driven App — Procurement & IT Workspace	Structured management of catalogue, requests, approvals and assignments.
Data	Microsoft Dataverse	Relational records, choice fields, lookups and centralized business data.
Automation	Power Automate	Approval orchestration, notifications and scheduled renewal checks.
Connectors	Dataverse, Approvals, Outlook / Office 365	Read/write business records and communicate decisions/reminders.
Analytics (companion capstone)	Power BI Subscription Creep & Hidden Cost Radar	Analyse recurring costs, license utilization and upcoming renewals.

4.1 Design principles

- Dataverse is the system of record; the apps are different interfaces over the same business data.
- Use lookup relationships rather than duplicating related record names wherever possible.
- Keep employee interactions simple and move administrative record management to the model-driven app.
- Use flows for asynchronous approvals, notifications and scheduled monitoring.
- Separate interface visibility from actual authorization: hiding a button is not security.
- Build the smallest complete lifecycle first, then add polish and optional enhancements.

5. Dataverse data model

Four core Dataverse tables form the relational data model. Software Catalog also contains License Duration (Months), used by provisioning to calculate an assignment-specific renewal date.

Table	Purpose	Key columns / fields
Software Catalog	Master catalogue of requestable products.	Software Name (primary name), Vendor, Category (Choice), Annual Cost (Currency), License Duration (Months) (Whole Number; default 12), Requires Approval (Yes/No), Is Available (Yes/No), Description.
Software Requests	One record per employee request.	Request Number (primary name), Requester Email, Department (Choice), Software (lookup), Business Justification, ReqStatus (Choice), Submitted On, Decision Date, Decision Comments.
Approvals	Decision record associated with a request.	Approval Name (primary name), Software Request (lookup), Approver Email, Decision (Choice), Comments, Decision Date.
License Assignments	Tracks allocation and renewal of a software license.	Assignment Name (primary name), Software (lookup), Assigned User Email, Software Request (lookup), Start Date, Renewal Date (Date Only), Annual Cost, Status (Choice), 30-Day Reminder Sent, 7-Day Reminder Sent, Overdue Reminder Sent.

5.1 Relationships

Parent table	Relationship	Child table	Meaning
Software Catalog	1:N	Software Requests	A software product can be referenced by many requests.
Software Requests	1:N	Approvals	A request can have one or more approval records.

Software Catalog	1:N	License Assignments	A product can have many license assignments.
Software Requests	1:N	License Assignments	The implemented provisioning policy creates one assignment per request. The lookup preserves the audit trail to the originating request.

In Dataverse, the many side of a one-to-many relationship is implemented through a lookup column on the child table. Creating the relationship manually still results in a lookup-backed relationship.

5.2 Choice fields

Table / column	Options confirmed in project
Software Catalog.Category	Productivity, Development, Design, Analytics, Collaboration, Security
Software Requests.Department	IT, Finance, HR, Marketing, Operations
Software Requests.ReqStatus	Draft, Submitted, Pending Approval, Approved, Rejected, Provisioned
Approvals.Decision	Pending, Approved, Rejected
License Assignments.Status	Available, Assigned, Revoked

Dataverse Choice values are stored as numeric option values. The Submitted option was used in Power Automate with the numeric value 120970001. Other option values should be taken from the actual environment rather than guessed.

6. Canvas App — Software Request Portal

The Canvas App is the employee-facing experience. It is designed for quick discovery and submission rather than exposing employees to the full Dataverse schema.

6.1 Confirmed screens and controls

Screen	Controls	Purpose / status
scrCatalog	lblTitle, txtSearch, galSoftware, btnRequest; My Requests navigation	Catalogue display, search and selected-software navigation. Confirmed as part of the built employee flow.
scrRequest	lblRequestTitle, lblSelectedSoftware, ddDepartment, txtJustification, btnSubmit, btnCancel	Request submission form. Created and submission tested successfully.
scrMyRequests	lblMyRequestsTitle, btnBackCatalog, galMyRequests, ddStatusFilter, btnRefreshRequests	Personal request tracking. User reported this screen was completed.

6.2 Key behaviours implemented

- Selecting a catalogue item stores the selected Dataverse record in varSelectedSoftware and navigates to scrRequest.
- The request screen displays the selected software name.
- Department is populated from the Dataverse Department Choice using Choices('Software Requests'.Department).
- The justification field is multiline.
- The submit action validates required values and uses Patch() to create a Software Requests record.

- The submission stores a generated request number, User().Email, department, software lookup, justification, Submitted status and submission timestamp.
- The successful submission was verified in Dataverse; all fields were reported correct and ReqStatus was Submitted.
- The My Requests gallery filters records by the signed-in user's email and supports status filtering and refresh.

6.3 Important implementation note

The Canvas submission path is implemented and was verified against Dataverse. For production hardening, the Patch result should still be captured and errors surfaced explicitly with IfError() or Errors(); this is a recommended improvement rather than a prerequisite for demonstrating the capstone workflow.

6.4 Final Canvas App scope

- The employee-facing Canvas App provides catalogue browsing, search, request submission and personal request tracking. Administrative catalogue and request maintenance is handled in the model-driven Procurement & IT Workspace, avoiding unnecessary duplication of admin features in the employee portal.
- The request submission flow validates required values, writes the request to Dataverse and returns the employee to the catalogue after successful submission.
- The My Requests screen filters records by the signed-in user's email and supports status filtering and refresh.
- The Canvas App is an interface, not an authorization boundary. Dataverse security roles and record privileges remain the correct enforcement mechanism.
- The app can be further hardened with explicit Patch error handling, loading indicators and duplicate-click prevention.
- A dedicated request-detail screen is an optional enhancement if additional time is available.

7. Model-driven App — Procurement & IT Workspace

The model-driven app is the administrative experience. It reuses the same Dataverse tables and is intended for IT/procurement users who need searchable lists, record forms and structured maintenance.

7.1 Confirmed work

- Created a model-driven app named Procurement & IT Workspace inside the Software Lifecycle Manager solution.
- Added all four Dataverse tables: Software Catalog, Software Requests, Approvals and License Assignments.
- Created custom Software Requests views named Pending Requests and Recently Approved.
- Pending Requests is intended to include ReqStatus = Submitted OR ReqStatus = Pending Approval.
- Recently Approved is intended to include ReqStatus = Approved.
- The user confirmed that the views were created and then made available in the model-driven app view selector.

7.2 Final administrative workspace

Area	Recommended work	Acceptance criteria
Software Catalog main form	Software Catalog form	Product, commercial and availability details are grouped for IT maintenance.
Software Requests main form	Software Requests form	Request, requester, decision and audit information are organized into sections.

Approvals main form	Approvals form	Linked request, approver, decision, comments and decision date are available for review.
License Assignments main form	License Assignments form	Software/user, assignment dates, renewal information and status are grouped for lifecycle management.
Views	Operational views	Pending Requests, Recently Approved and license-oriented views help administrators locate work.
Business rules	Validation	Choice values and required lookup/date fields are configured to support consistent records.
Security	Access governance	Dataverse roles and record-level access should be validated before production deployment.

The model-driven app provides table views and forms for IT/procurement administration. The Software Requests view selector includes Pending Requests and Recently Approved. License Assignments also has operational views, including Active Licenses and renewal-oriented views such as Expiring Soon / Next Month where configured. Forms are organized into sections so administrators can inspect related information, dates, status and decision history.

8. Power Automate flows

8.1 Flow 1 — SLM – Software Request Approval

Purpose: automate the approval lifecycle when a new software request is submitted.

Element	Configuration / behaviour
Trigger	Microsoft Dataverse — When a row is added, modified or deleted; configured for Added; table Software Requests; organization scope.
Gate condition	ReqStatus equals the Submitted Choice option. The numeric option value used was 120970001.
Create approval row	Add a new row in Approvals, linked to the triggering Software Request, with approver email and Decision = Pending.
Approval action	Start and wait for an approval; Approve/Reject — First to respond; title and details include request context.
Outcome condition	Outcome equals the literal text Approve.
If yes	Update Software Requests to Approved; update the created Approvals row to Approved; record comments and decision date.
If no	Update Software Requests to Rejected; update the created Approvals row to Rejected; record comments and decision date.

Confirmed workflow design: Flow 1 was tested successfully for approval handling. Its approved branch was subsequently extended to retrieve the selected Software Catalog record, create a linked License Assignment, calculate the renewal date from License Duration (Months), update the original request to Provisioned, and send a provisioning confirmation. The assignment uses Status = Assigned and links to both the Software Catalog and original Software Request. A final environment run should be retained as evidence for the completed provisioning extension.

8.2 Flow 2 — SLM – License Renewal Watchdog

Purpose: monitor license assignments and alert the responsible person before renewal dates, while preventing the same milestone reminder from being sent repeatedly.

Element	Scheduled cloud flow; recurrence every 1 day.
Trigger	Dataverse List rows on License Assignments, selecting renewal date, assigned user, software, status and reminder flags.
Retrieve records	Apply to each over the List rows value array.
Loop	Process records whose custom License Assignment Status is Assigned and whose renewal date is relevant to a reminder window.
Eligibility	When renewal is 30 days away and 30-Day Reminder Sent = No, email both recipients and then set the flag to Yes.
30-day reminder	When renewal is 7 days away and 7-Day Reminder Sent = No, email both recipients and then set the flag to Yes.
7-day reminder	When renewal date is before today and Overdue Reminder Sent = No, send one overdue email to both recipients and set the flag to Yes.
Overdue reminder	Assigned user and administrator; overdue reminder frequency is once.

The renewal watchdog is designed as a scheduled cloud flow that runs daily, lists License Assignments, iterates through the returned rows and processes eligible Assigned records. The configured policy checks 30-day, 7-day and overdue milestones; sends reminders to both the assigned user and administrator; and updates the corresponding reminder-sent flags after successful email delivery. Overdue notification is one-time. Renewal Date is a Dataverse Date Only column, so comparisons must consistently use date-only values and account for the flow's timezone.

8.3 Flow 2 implementation safeguards

- Use the actual Renewal Date logical name from Dataverse column properties; the renewal date is a Date Only field.
- Filter eligible License Assignments to the custom Status choice Assigned.
- Use date-window comparisons rather than exact equality so a missed daily run does not permanently miss a reminder.
- Use consistent date-only comparisons and account for UTC versus local date boundaries.
- After a reminder email succeeds, update the matching License Assignments row using its unique Row ID and set the matching reminder flag to Yes.
- Keep the reminder flags false when the email action fails; do not mark a reminder as sent before successful delivery.
- The reminder policy is daily evaluation, 30-day and 7-day milestone reminders, and a one-time overdue reminder. Both the assigned user and administrator are recipients.

8.3 Automatic license provisioning within Flow 1

The approved branch of the request approval flow performs provisioning after the approval decision is recorded. It retrieves the Software Catalog row using the Software lookup GUID, reads Annual Cost and License Duration (Months), creates one License Assignment, links it to the original Software Request and Software Catalog records, assigns the requester as the license holder, sets Status = Assigned, calculates Renewal Date from Start Date plus the catalog duration, updates ReqStatus to Provisioned and sends a confirmation email. The agreed provisioning policy is one assignment per request and a default catalog duration of 12 months, with the catalog value controlling the actual term.

Step	After the approval outcome is Approve, continue in the If yes branch of Flow 1.
1	Retrieve the Software Catalog row using the Software lookup GUID from the request trigger.

2	Create exactly one License Assignment linked to the Software Catalog and Software Request rows.
3	Populate requester email, Start Date, Annual Cost, Status = Assigned and Renewal Date = Start Date plus License Duration (Months) from the catalog.
4	Update Software Requests ReqStatus to Provisioned after assignment creation.
5	Send provisioning confirmation to the requester; prevent duplicate assignment creation on retries.
6	Record enough information to prevent duplicate assignment creation.

The provisioning action is part of the approved path in Flow 1 rather than a separate standalone Flow 3. The design should prevent duplicate assignment creation if a flow run is retried; the request-to-assignment relationship and a lookup check provide the basis for this safeguard.

9. Security, roles and governance

A role-based interface is useful, but it is not a security boundary. A Canvas App Visible property or a variable such as varIsAdmin only controls what the interface displays. Actual authorization must be enforced through Dataverse security roles, table privileges, ownership/team access and appropriate row-level scope.

Persona	Intended access
Employee	Read available Software Catalog records; create requests; read only their own requests and relevant decision information.
Approver / Manager	Read requests assigned or accessible to them; create/update relevant approval decisions.
IT / Procurement Admin	Manage software catalogue, review requests, manage approvals and create/update license assignments.
System administrator	Configure environment, solutions, security roles and application administration.

9.1 Security and governance considerations

- Assign Dataverse security roles for Employee, Approver and IT Administrator, with least-privilege table permissions.
- Test role assignments with separate user accounts where possible, not only the maker/admin account.
- Verify that employees can access only their own request records and cannot edit catalogue or license assignment data unless explicitly authorized.
- Confirm that direct Dataverse access is governed by privileges and ownership/team access, not merely by Canvas App filters or hidden buttons.
- Document any limitations imposed by the training tenant, including licensing and available connector permissions.
- For a production deployment, complete a formal security review and validate role-based and row-level access with test personas.

10. Testing and acceptance plan

Test ID	Scenario	Expected result
T01	Employee opens catalogue.	Available software products are displayed.
T02	Employee searches by software name.	Gallery filters to matching available products.

T03	Employee submits a complete request.	Dataverse Software Requests row is created with correct lookup, department, email, justification, Submitted status and timestamp.
T04	Employee submits with missing justification.	Validation message appears and no row is created.
T05	Employee opens My Requests.	Only their own requests appear.
T06	New request triggers Flow 1.	Approval record is created and approval request is sent.
T07	Approver approves.	Request and Approval records both become Approved; comments/date are recorded.
T08	Approver rejects.	Request and Approval records both become Rejected; comments/date are recorded.
T09	Flow 2 finds a license due in 30 days.	One 30-day reminder is sent and its flag becomes Yes.
T10	Flow 2 runs again for same assignment/milestone.	No duplicate 30-day reminder is sent.
T11	License becomes due in 7 days or overdue.	Correct milestone reminder is sent once and matching flag is updated.
T12	IT provisions an approved request.	One linked License Assignment is created and request becomes Provisioned.
T13	Employee attempts admin action.	Dataverse security prevents unauthorized write even if the UI is bypassed.
T14	Model-driven views are opened.	Pending Requests and Recently Approved show only matching statuses.

11. Remaining work and priorities

Priority	Work item	Status
P0	Complete Flow 2 date expressions, reminder conditions, emails, flag updates and test.	Configured as the daily 30-day / 7-day / one-time overdue reminder workflow; retain successful run evidence.
P0	Verify Flow 1 outcome records and comments in Dataverse for both approval and rejection.	Approval and provisioning workflow implemented; retain Approved and Rejected run evidence.
P1	Polish model-driven forms and views for all four tables.	Core forms, views and navigation configured.
P1	Configure and test Dataverse security roles and record-level access.	Production-hardening validation recommended.
P1	Build Flow 3 license provisioning.	Implemented within Flow 1 approved branch; no separate Flow 3 required.
P1	Harden Canvas submit logic with error handling and duplicate-submit prevention.	Recommended remaining
P2	Add Admin Workspace in Canvas App for catalogue and request management.	Administrative work is handled through the model-driven app.
P2	Add business rules and form-level validation.	Optional refinement.
P2	Prepare final sample data, demo script, screenshots and presentation.	Prepare/retain final screenshots, run history and demo evidence.

12. Recommended final build sequence

1. Run the end-to-end request journey: submit, approve, create linked assignment, update request to Provisioned and send confirmation.
2. Verify both Approved and Rejected paths, including Approval records, comments and decision dates.
3. Test the renewal watchdog with controlled records due in 30 days, 7 days, overdue, future-dated and revoked/inactive states.
4. Run the scheduled flow again to confirm that sent flags prevent duplicate milestone emails.
5. Review model-driven forms, views and navigation for all four tables.
6. Validate Dataverse security roles and record access with employee and administrator personas.
7. Capture screenshots and run history for the final demonstration and evidence pack.

13. Final demonstration storyline

A five-to-seven-minute demonstration can tell one coherent story:

8. Show the employee opening the Canvas App and browsing the Software Catalog.
9. Select a product, explain the business need and submit a request.
10. Show the new Software Requests record in Dataverse or My Requests.
11. Show Flow 1 creating an Approval record and sending the manager an approval request.
12. Approve or reject the request and show both Dataverse records updating.
13. Open the model-driven Procurement & IT Workspace and show the request views and administrative forms.
14. Show an approved request automatically creating one linked License Assignment, updating the request to Provisioned and sending the requester a confirmation.
15. Show the daily renewal watchdog evaluating a license and sending the correct 30-day, 7-day or one-time overdue reminder to the assigned user and administrator, with the corresponding flag updated.

14. Risks, assumptions and limitations

Risk / assumption	Mitigation
Dataverse, Approvals and Outlook connector availability depends on training-tenant licensing and permissions.	Validate connectors and environment access before building optional features.
Choice fields may appear as labels or numeric values in Power Automate.	Use actual option values from the environment; do not guess numeric values.
Lookup updates require a related row GUID, not a display name.	Use trigger Row ID or Add a new row output ID as appropriate.
Scheduled flow date comparisons can be affected by time zones.	Use date-only comparisons consistently and test near midnight/time-zone boundaries.
Reminder flags alone do not prevent duplicates if a flow run partially fails.	Update flags only after successful send; consider run concurrency and retry behaviour.
UI-only admin checks are bypassable.	Enforce access with Dataverse security roles and row-level privileges.
Sample prices and license data are illustrative.	Label demo data as fictional and avoid presenting it as vendor pricing.

15. Conclusion

Software Lifecycle Manager is a connected Power Platform business process rather than a collection of isolated forms. It combines an employee-facing Canvas App, a model-driven IT/procurement workspace, a relational Dataverse foundation, automated approval and provisioning, and scheduled renewal monitoring. The design links each License Assignment to its originating request and software catalogue record, providing a traceable path from request through approval and allocation to renewal.

The capstone demonstrates why Canvas and model-driven apps serve different personas, why Dataverse lookups are important for traceability, and how Power Automate can coordinate approvals, provisioning and recurring reminders. It also provides a foundation for the companion Power BI Subscription Creep & Hidden Cost Radar, which can analyse recurring cost and renewal exposure.

Appendix A — Suggested project vocabulary

Term	Meaning in this project
Software catalogue	List of software products employees may request.
Software request	An employee's request to acquire or access a product.
Approval	A recorded decision associated with a request.
License assignment	A record that a license has been allocated to a user.
Provisioning	The operational step of assigning/access-enabling the approved software.
Renewal monitor	Scheduled process that checks upcoming and overdue renewal dates.
Choice	Dataverse option-set column used for controlled status/category values.
Lookup	Dataverse relationship column that references another table's row.